BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent` , create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')` .
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states` .
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)` . For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost` . If `current_pos == goal_pos` , return the `path_taken` . Otherwise, add `current_pos` to `reached_states` .
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent` .
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos` . Call your `astar_search` method and store the returned list of actions in `self.plan` .

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()` . Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

IT24103352

UCS prioritizes which node to explore based solely on the actual path cost from the start node to the current node, denoted as g(n). It always expands the node with the lowest g(n). A* Search, on the other hand, uses an informed approach. It prioritizes nodes based on an evaluation function f(n) = g(n) + h(n), which combines the actual path cost from the start node, g(n), and the estimated cost to reach the goal, h(n) (the heuristic). It expands the node with the lowest combined cost, f(n).

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

An admissible heuristic must never overestimate the true cost to reach the goal. In a grid restricted to 4-way movement (Up, Down, Left, Right), the shortest possible path between two points (assuming no walls) is exactly the Manhattan distance. Since the addition of walls can only increase the actual path cost, the Manhattan distance will always be less than or equal to the true cost, making it admissible.

If the heuristic was NOT admissible (i.e., it sometimes overestimated the cost), A* would lose its guarantee of optimality. It might prematurely dismiss an optimal path thinking it is too costly and instead return a suboptimal, longer path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan distance would not be admissible in an 8-way movement grid. If the agent moves diagonally, it covers both horizontal and vertical distance in a single step (cost of 1). However, Manhattan distance would calculate this as two separate steps (cost of 2). Therefore, it would overestimate the true cost to the goal.

You should switch to the Chebyshev distance (also known as the chessboard distance). It takes the maximum of the absolute differences of their coordinates: max(|x1 - x2|, |y1 - y2|), which correctly estimates the minimum steps needed when diagonal movement is allowed and costs the same as cardinal movement. Alternatively, if diagonal movement costs more (e.g., √2), you could use Euclidean distance or a tailored Diagonal distance.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A much stronger and admissible heuristic for the multi-goal problem (like the Traveling Salesperson Problem on a grid) is to compute the Minimum Spanning Tree (MST) of the remaining unvisited food locations, including the agent's current position.

You treat the agent and all remaining food items as nodes in a graph, with the Manhattan distances between them as the edge weights. The sum of the edges in the resulting MST provides a tight lower bound on the cost to visit all remaining food items. This is a very informative heuristic that accounts for the spatial distribution of all the food, avoiding the trap of simply going to the closest food while moving away from the rest of the cluster.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

FACULTY OF COMPUTING